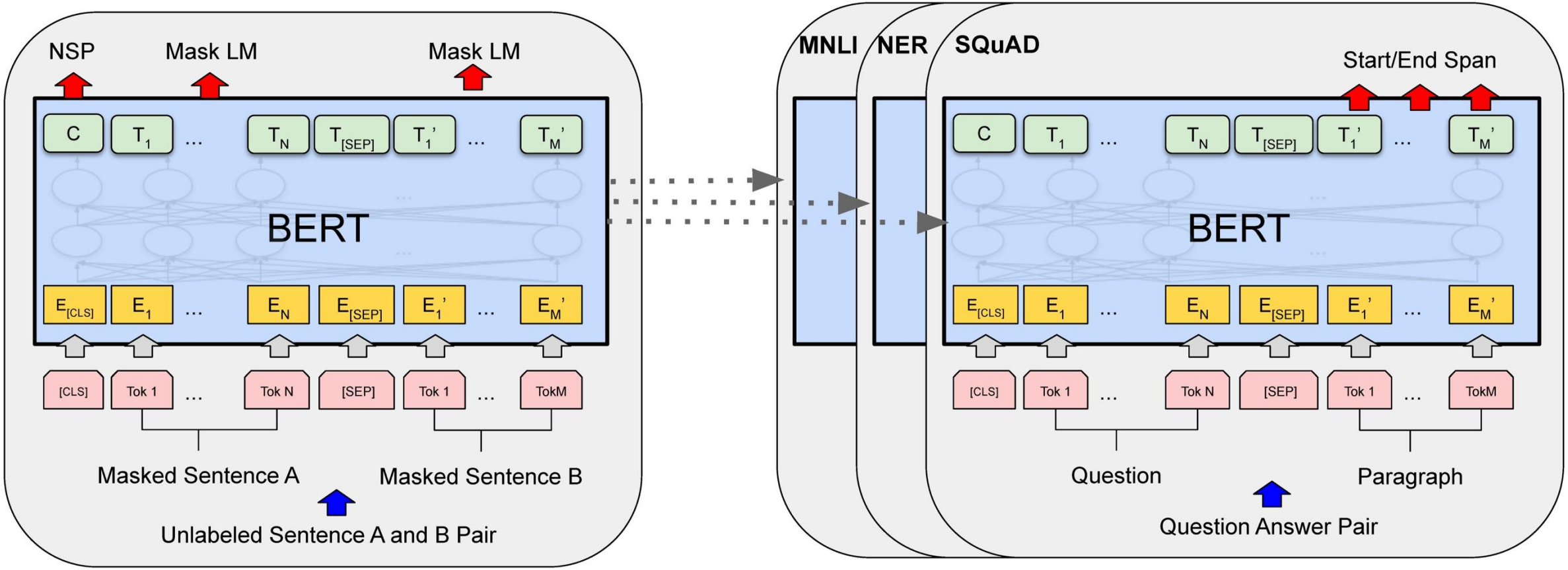
ECE 590
Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 13
02/24/2026

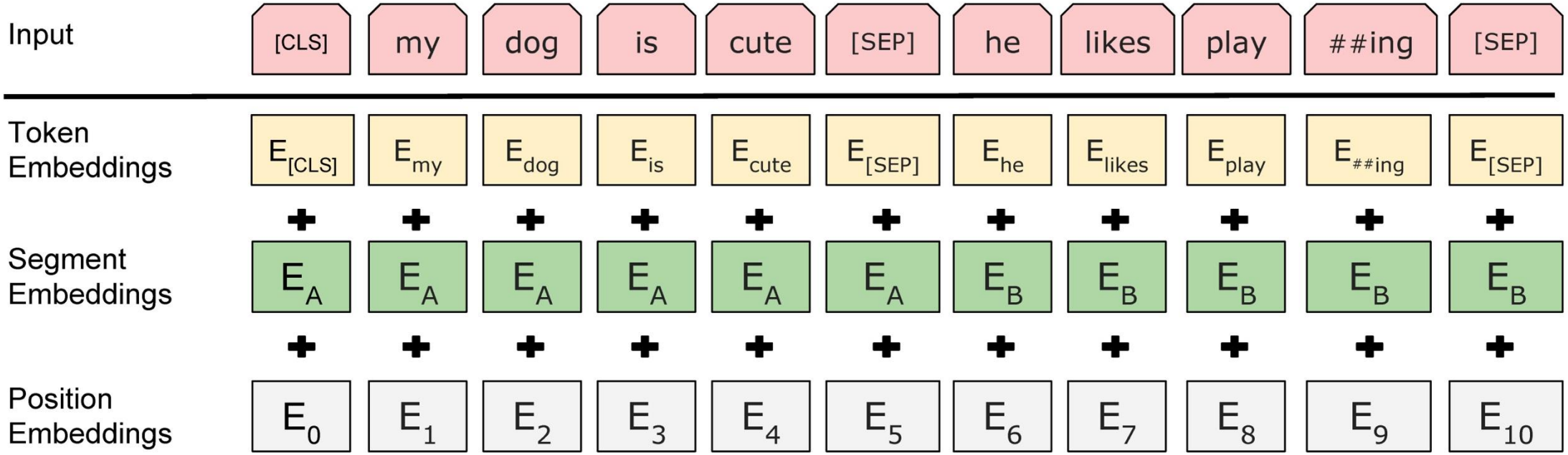
Technical Details of BERT



Pre-training

Fine-Tuning

Token Embeddings for Masked Tokens



- 15% of input tokens masked

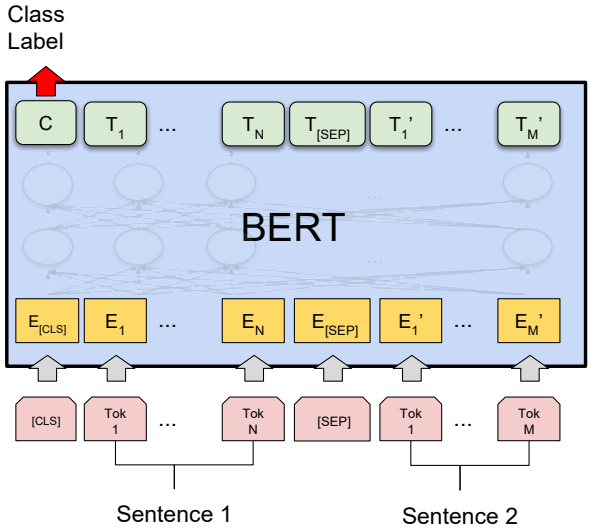
- Of these, for 80% a special [MASK] token used (learned)
- 10% of time put a random word at position of mask
- 10% put the original word in

Use output above masked word to predict original word

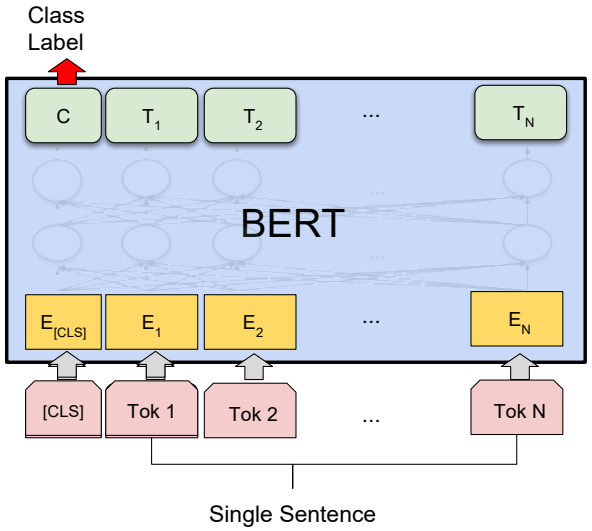
Learn to Classify When Pretraining

- BERT learns how to predict masked words based on surrounding words (not autoregressive)
- Bidirectional attention
- It also learns to classify, based on unlabeled data
- Next-sentence prediction (NSP)
- Effectively learns features of *sentences* that are predictive of other things
- Requires a learning of structure and meaning of language, along with prediction of masked words which also manifests an understanding of language

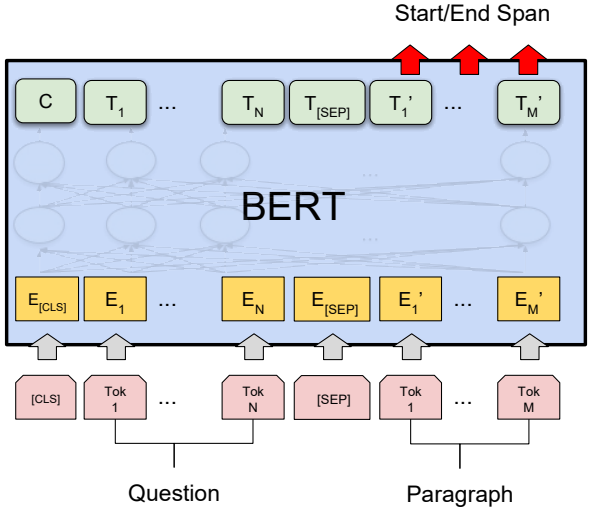
Down-Stream Tasks



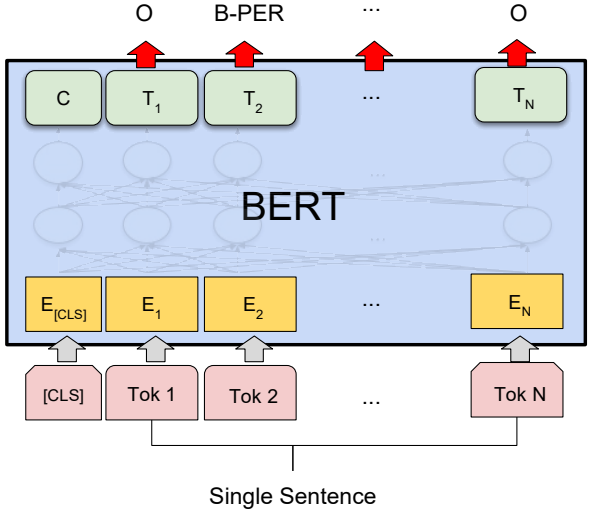
(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



(b) Single Sentence Classification Tasks: SST-2, CoLA

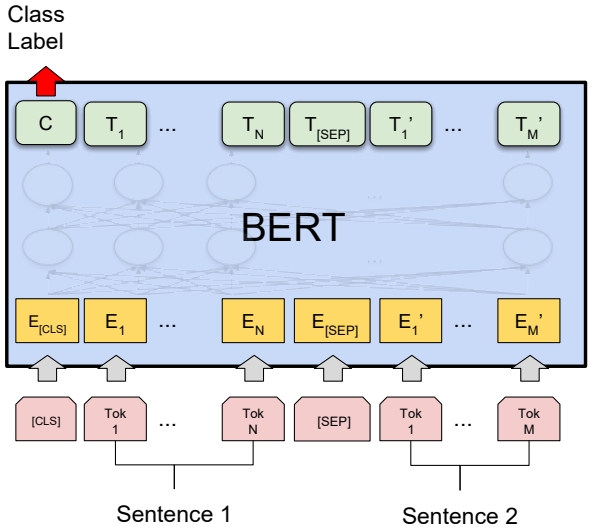


(c) Question Answering Tasks: SQuAD v1.1

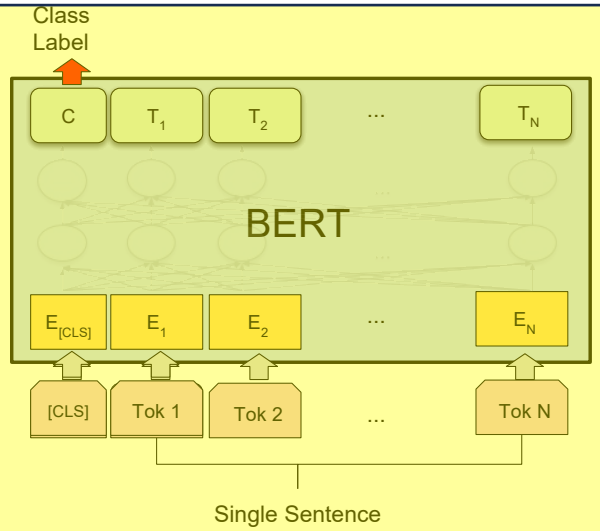


(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

Down-Stream Tasks

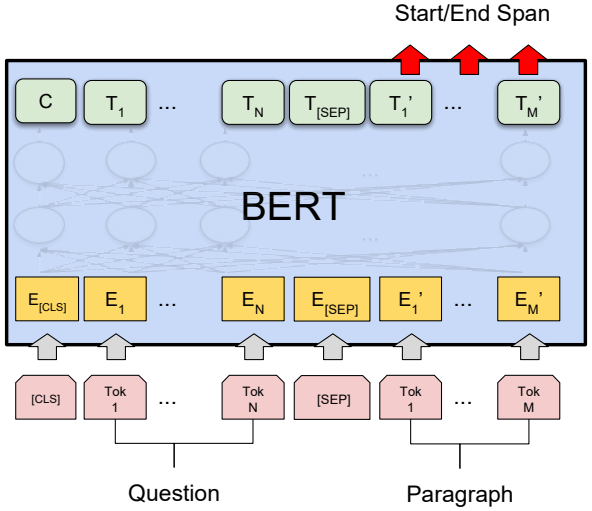


(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

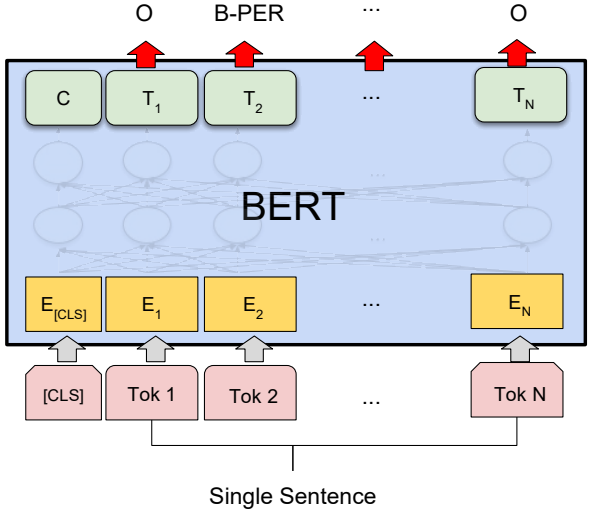


(b) Single Sentence Classification Tasks: SST-2, CoLA

Sentiment Analysis

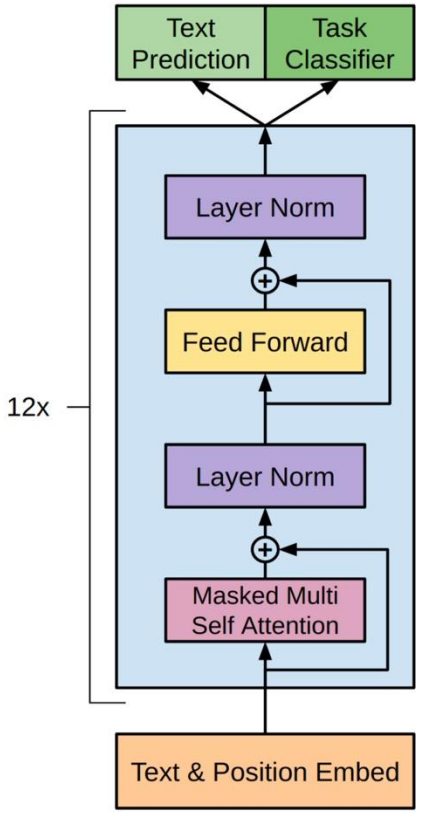


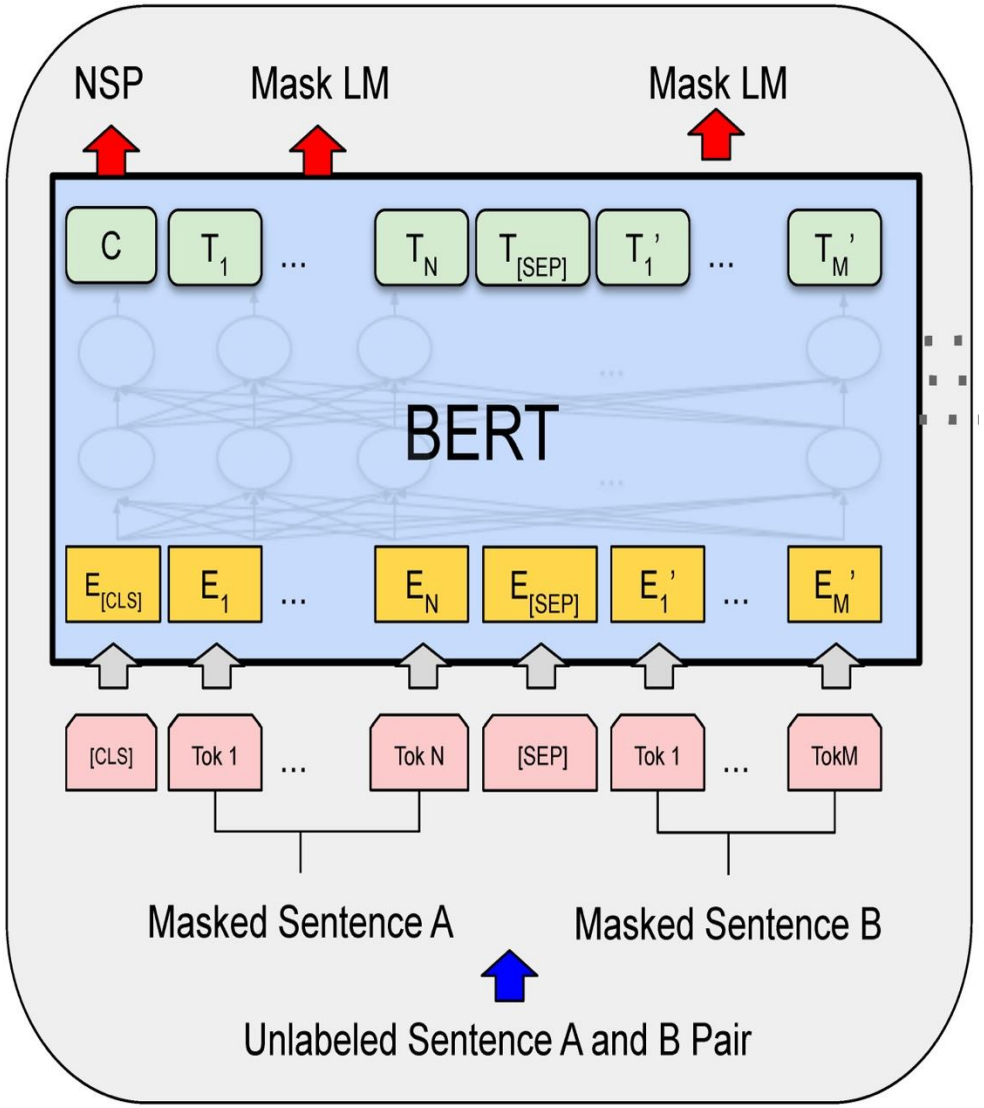
(c) Question Answering Tasks: SQuAD v1.1



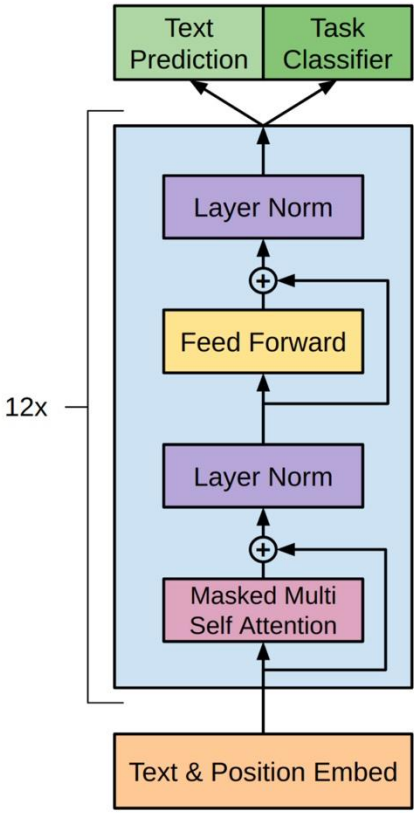
(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

Key Attribute of all LLMs

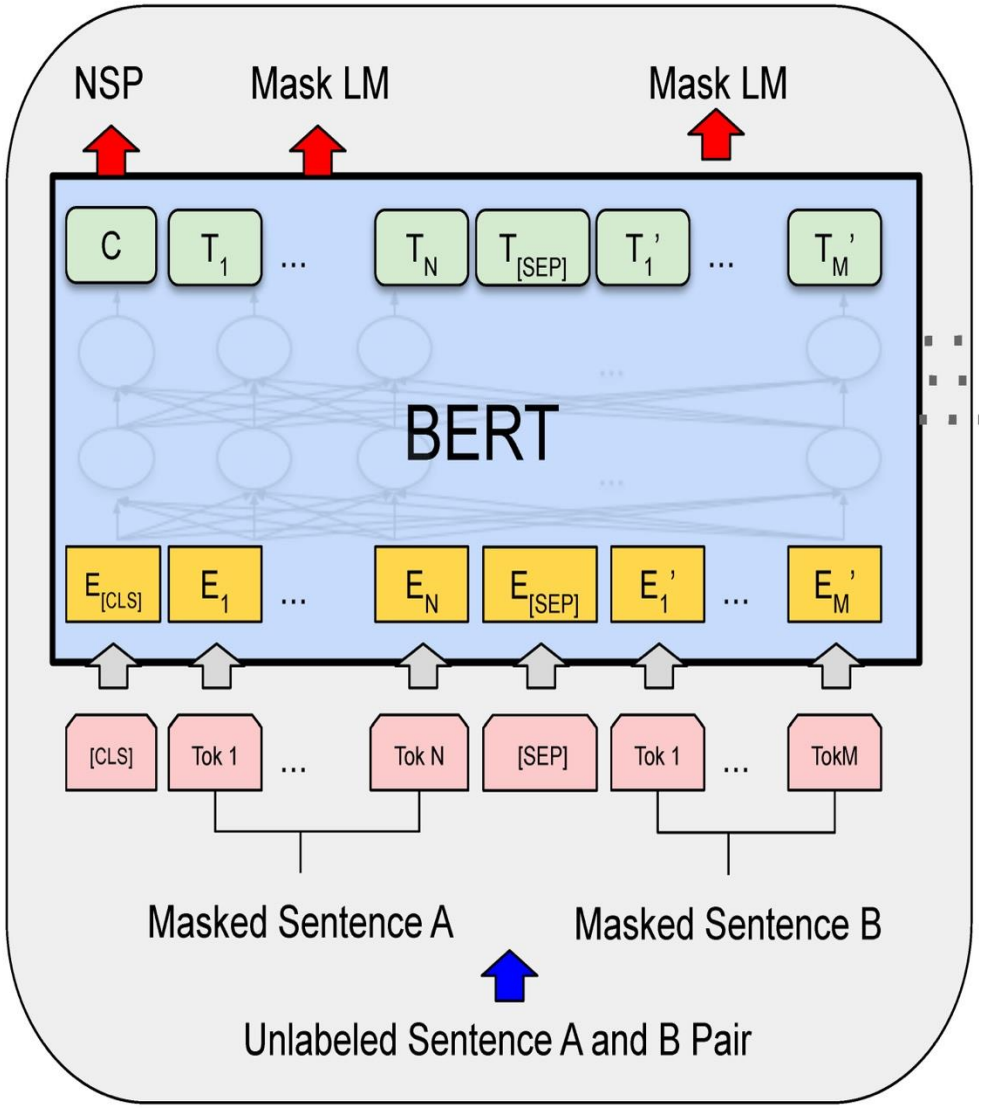




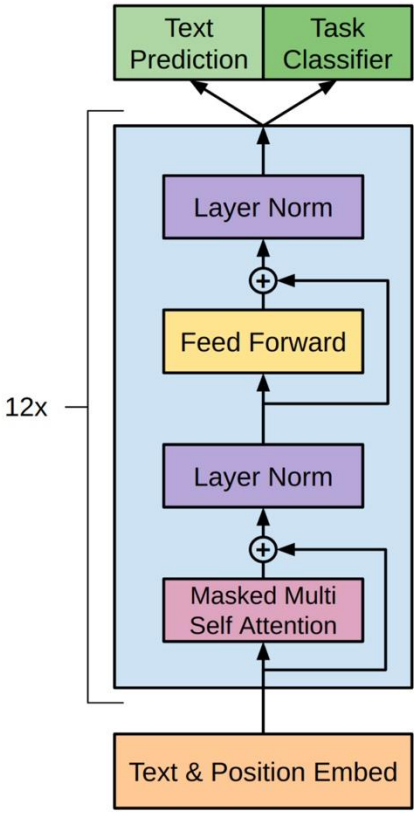
Key Attribute of all LLMs

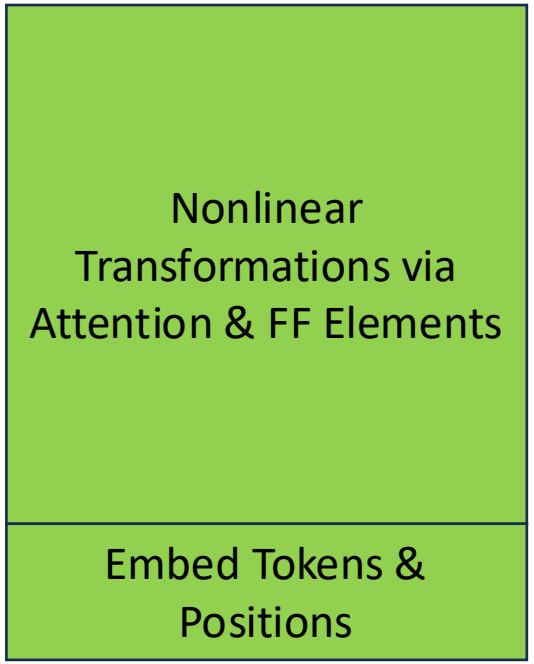


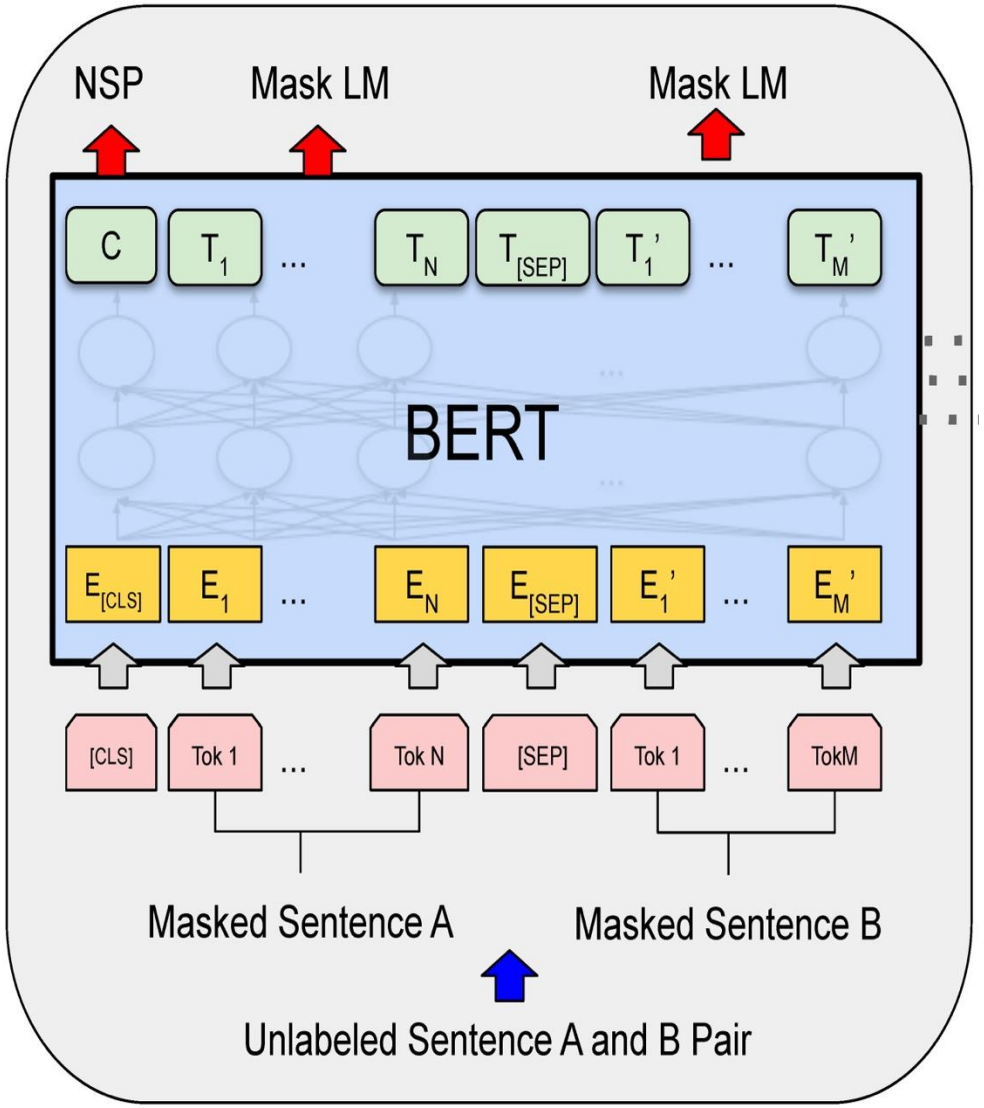
Embed Tokens & Positions



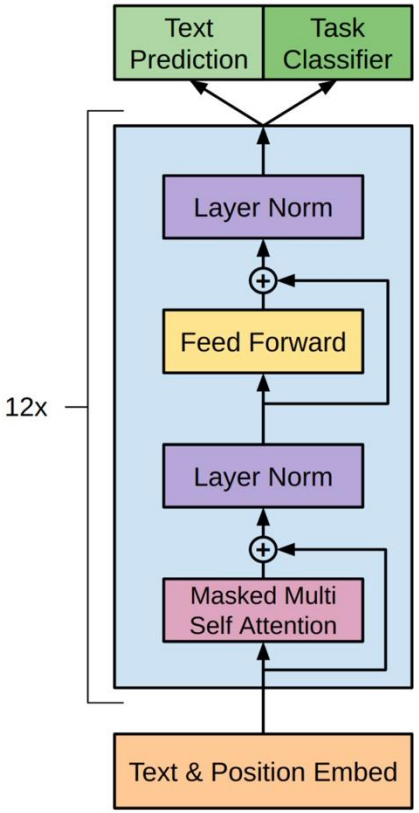
Key Attribute of all LLMs

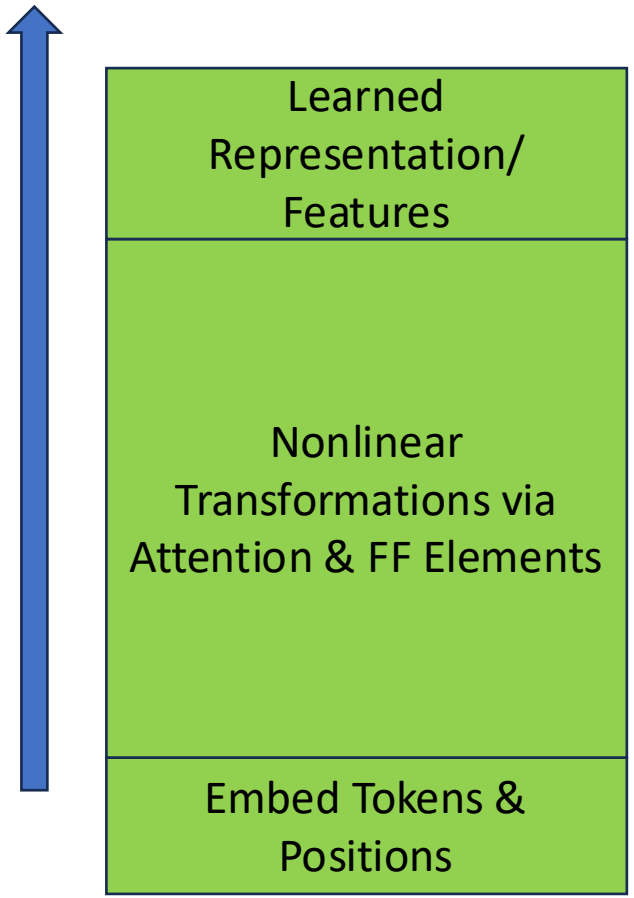


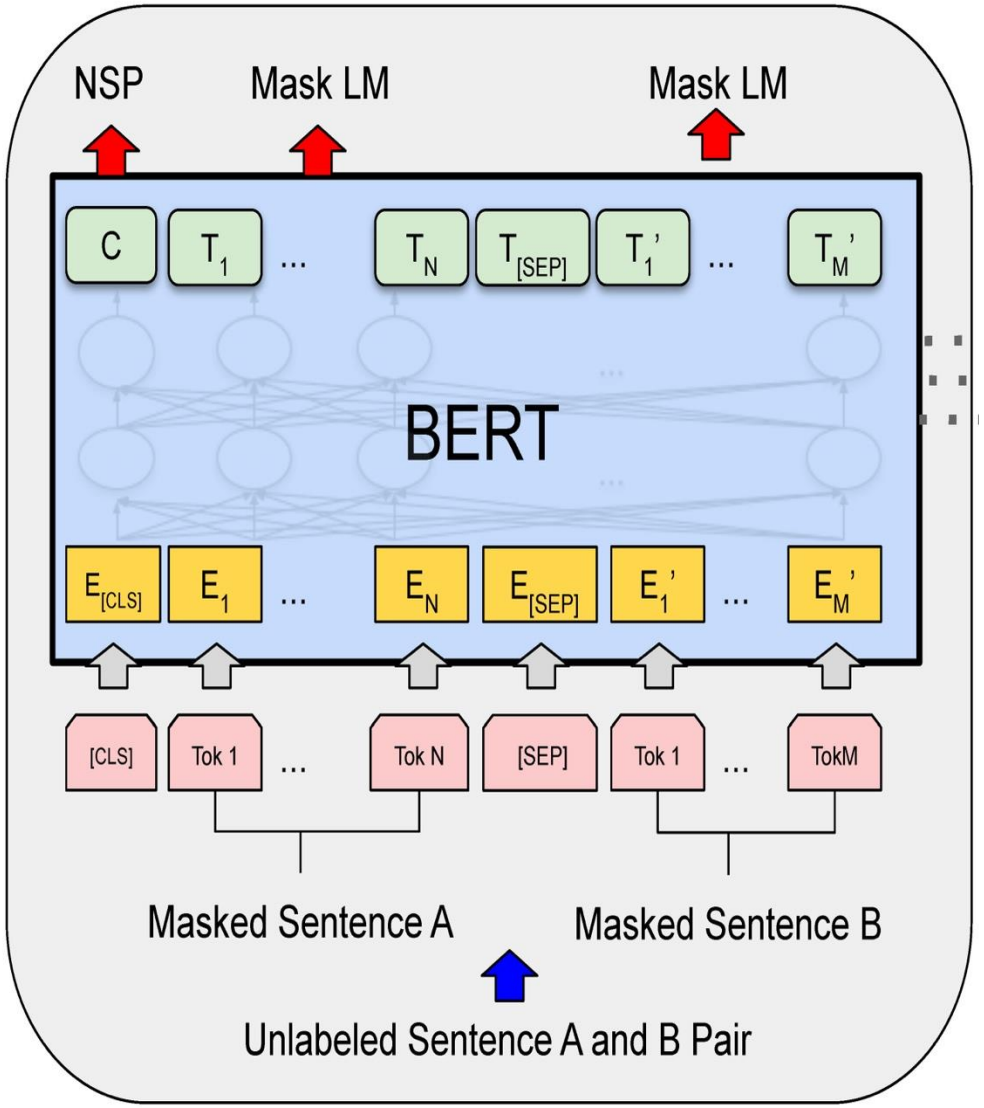




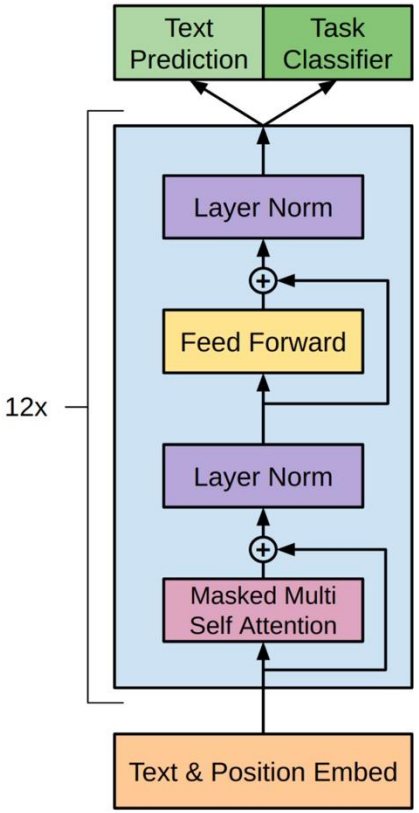
Key Attribute of all LLMs

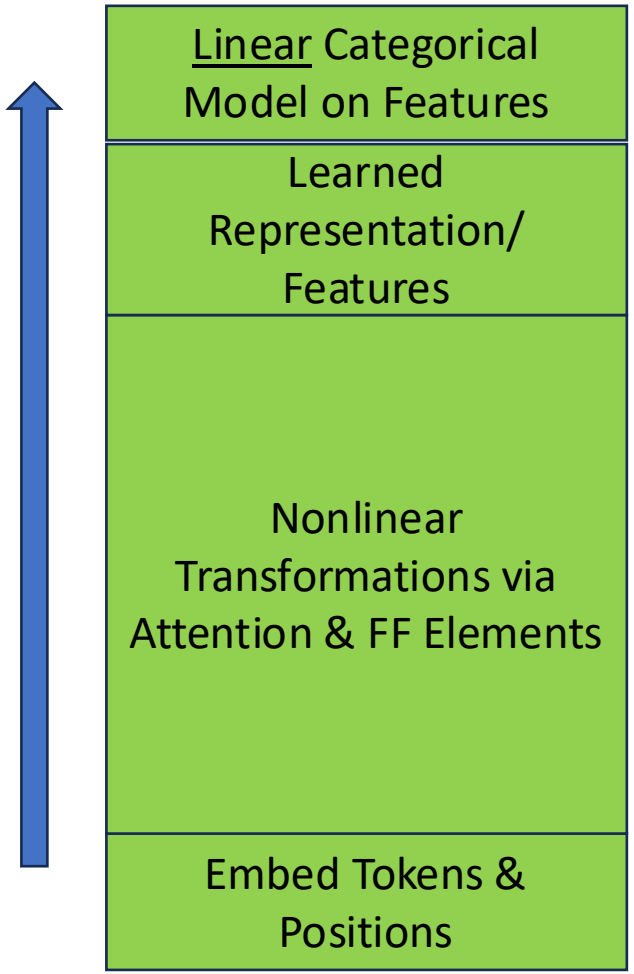


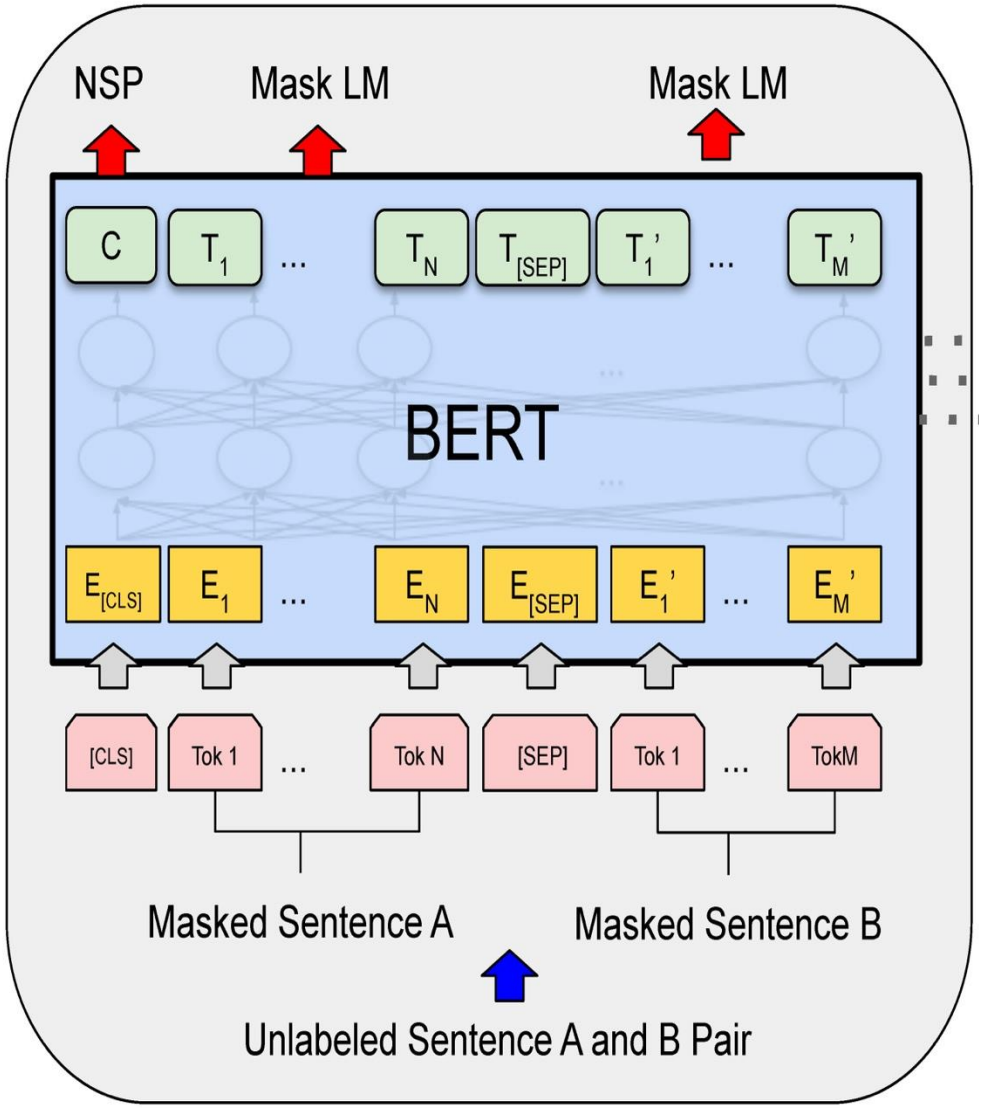




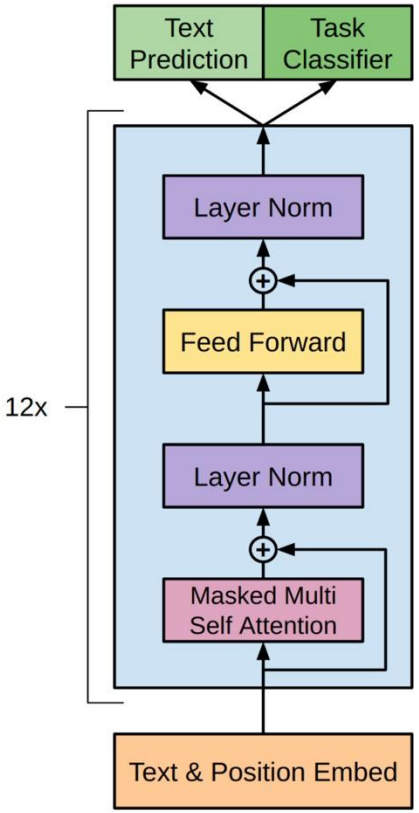
Key Attribute of all LLMs

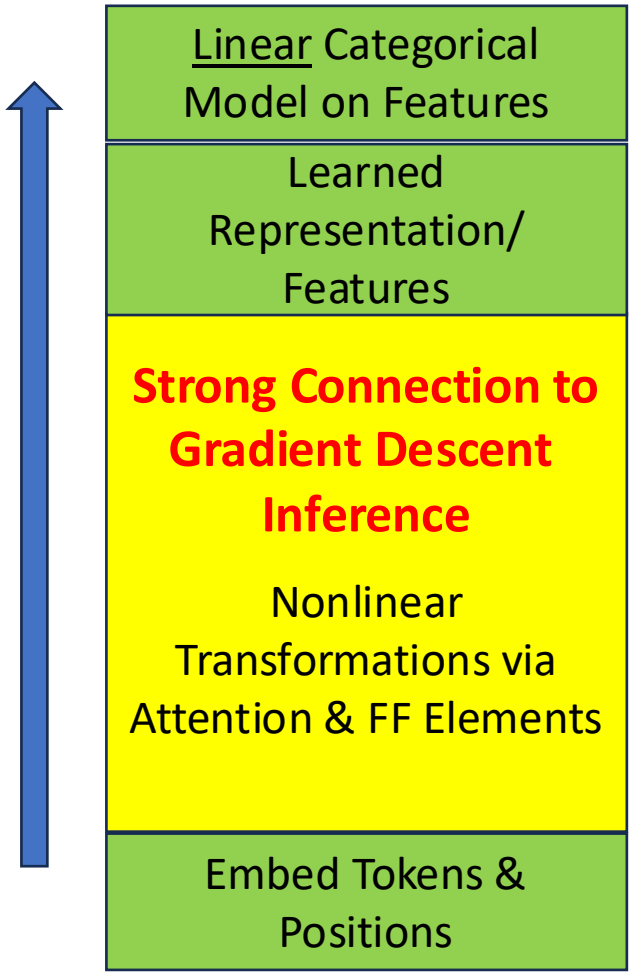


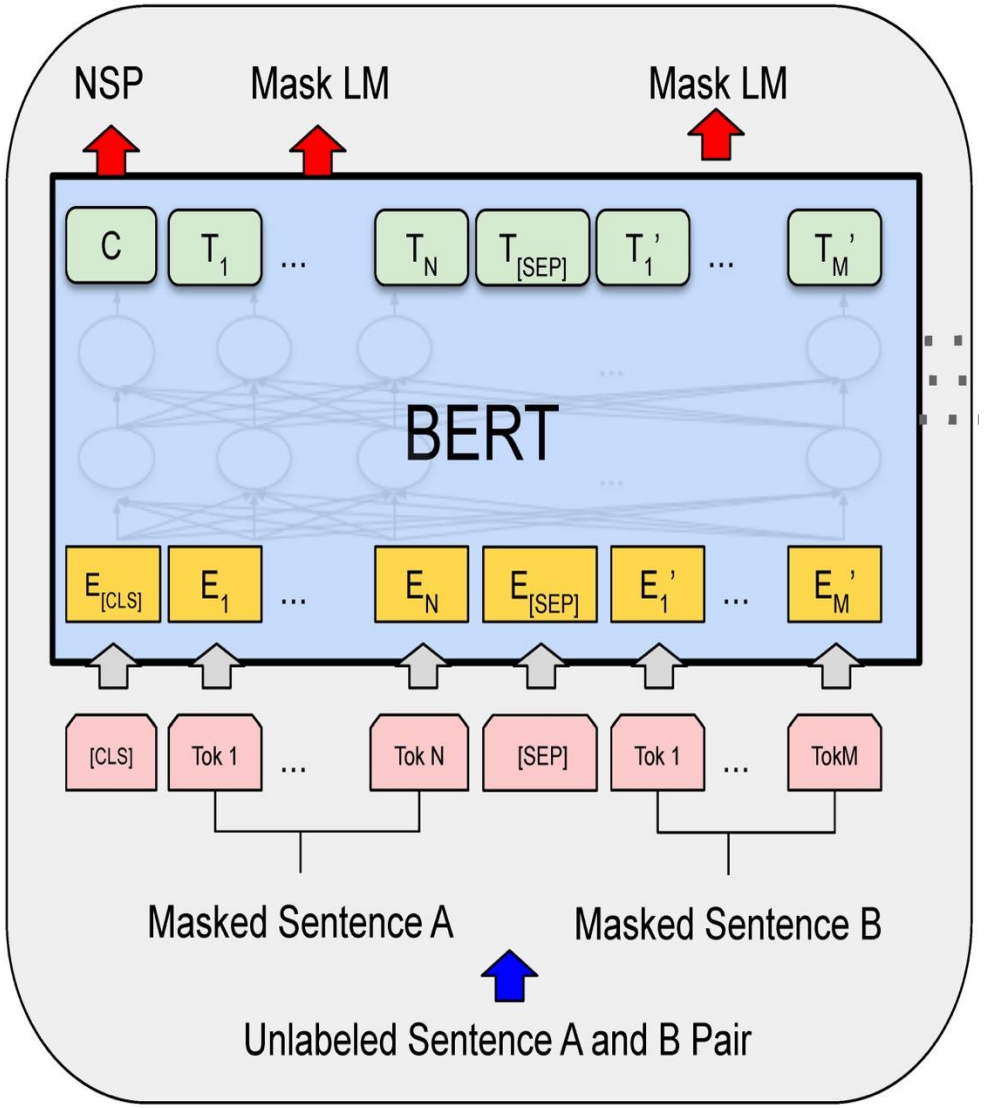




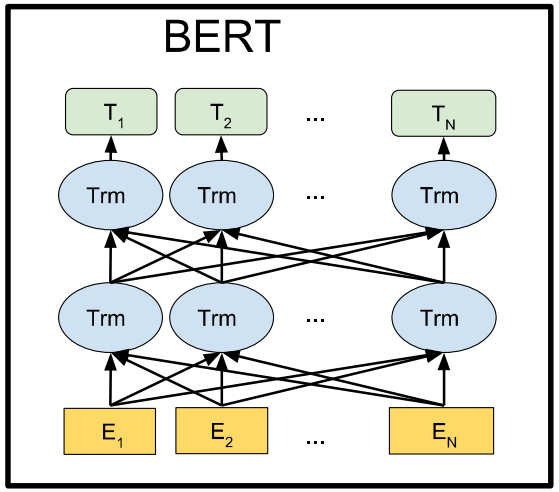
Key Attribute of all LLMs

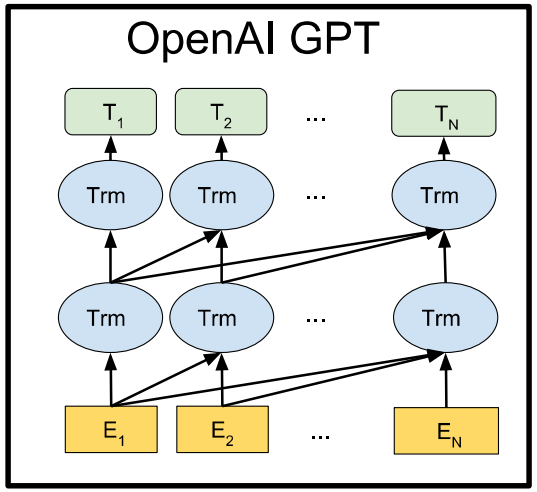


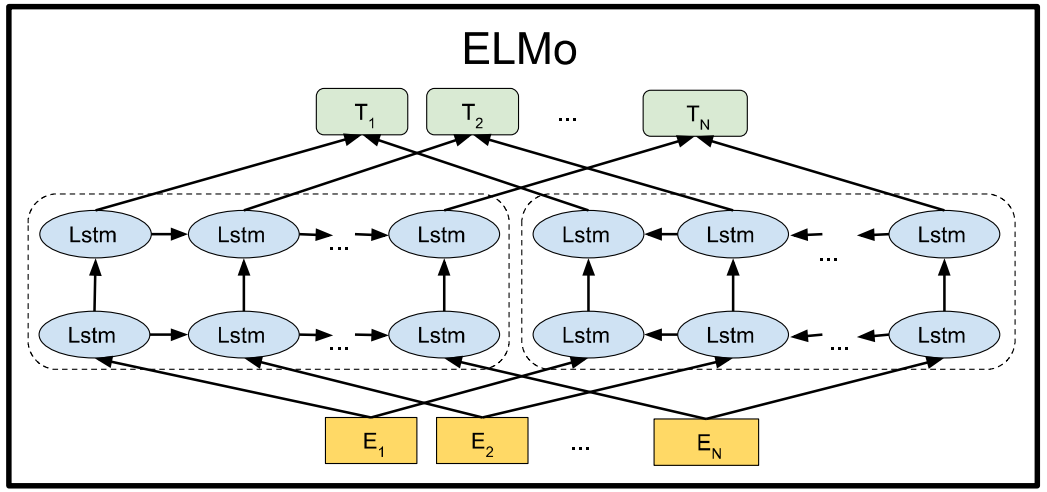




Different Types of Large Language Models

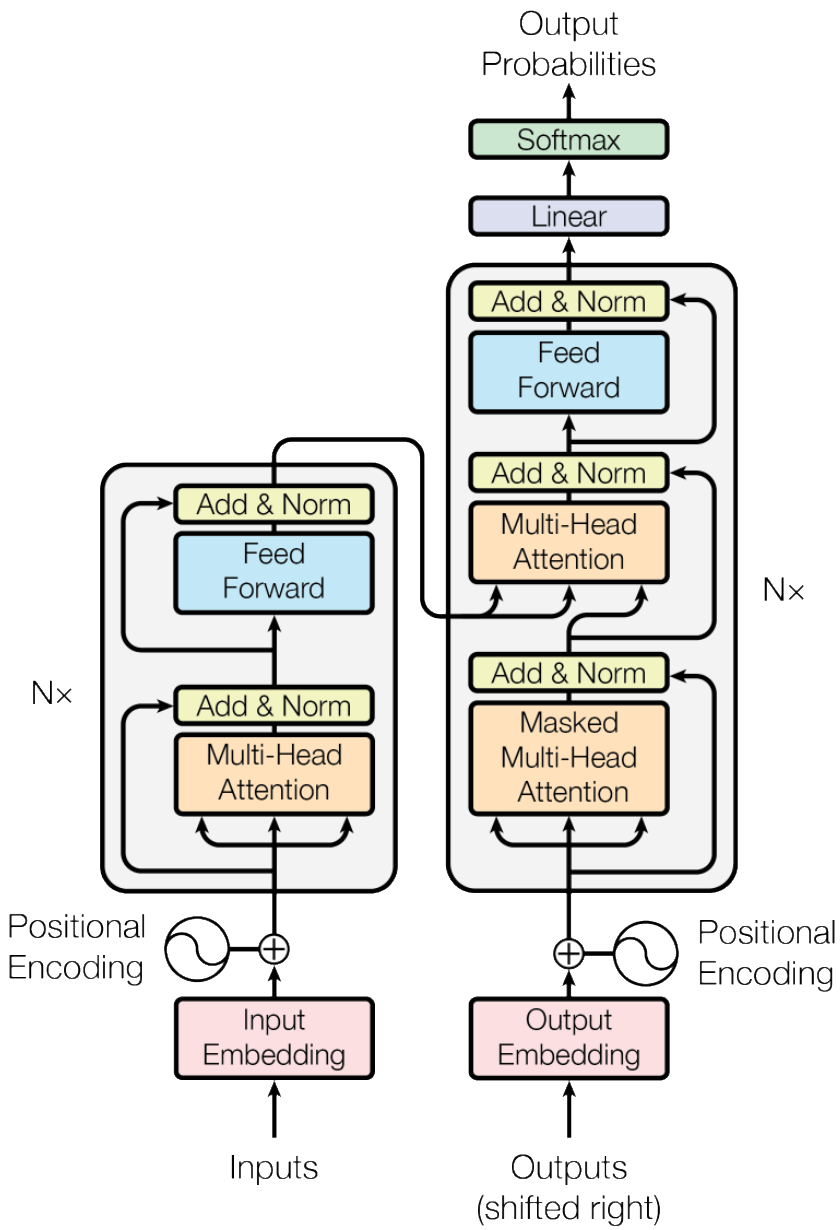






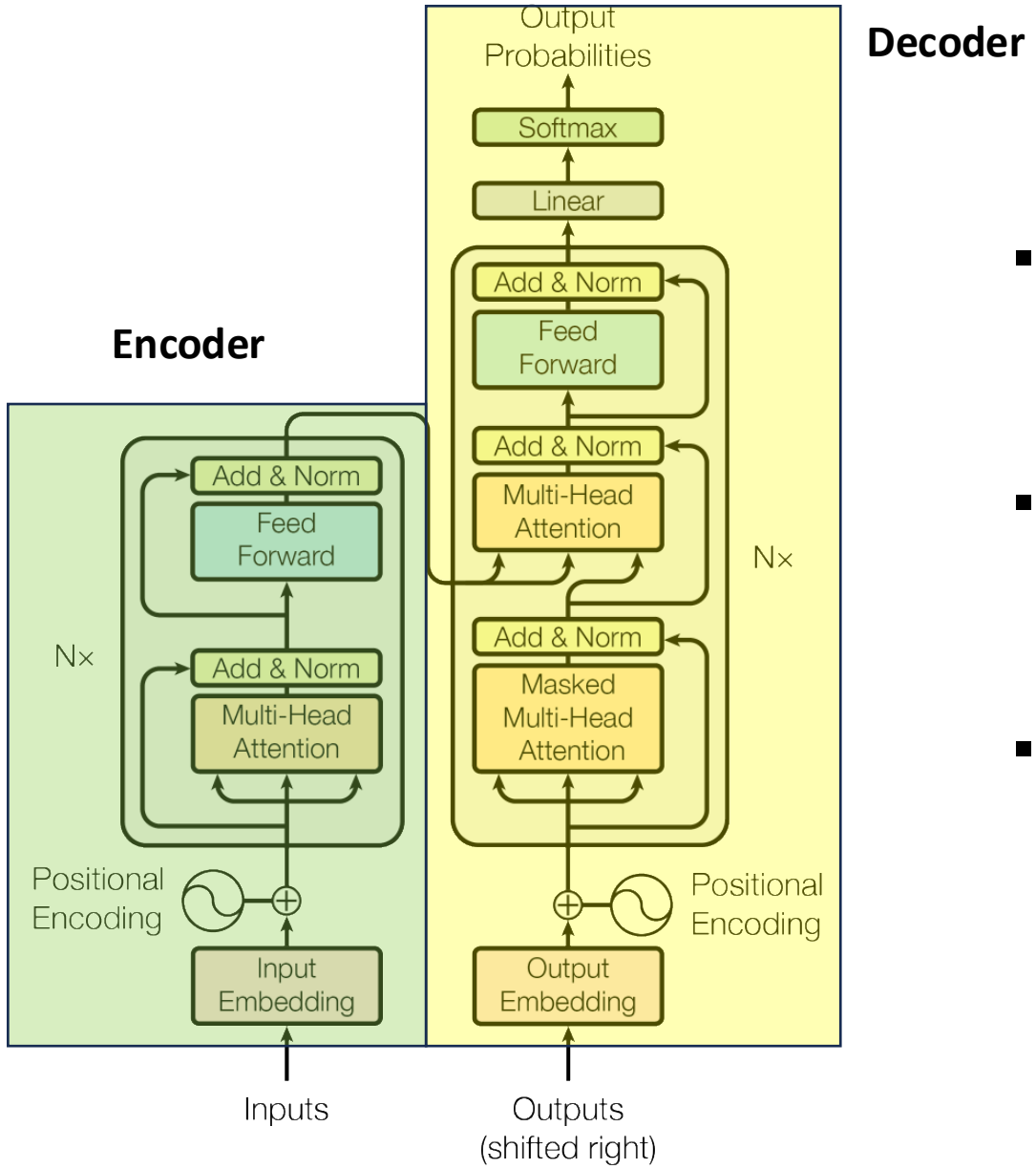
Based on Long Short-Term Memory (LSTM)
Technology that Preceded the Transformer

Original Transformer



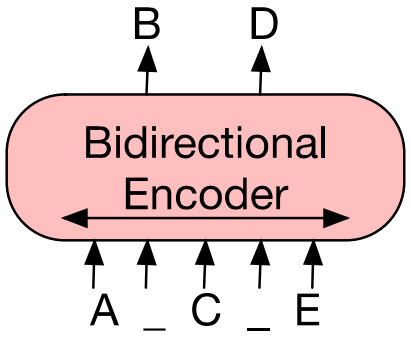
- Encode the prompt and generated text separately
- On the generation arm, perform cross attention with encoded prompt
- On generator, have self attention and cross attention layers, in addition to the FF element

Original Transformer

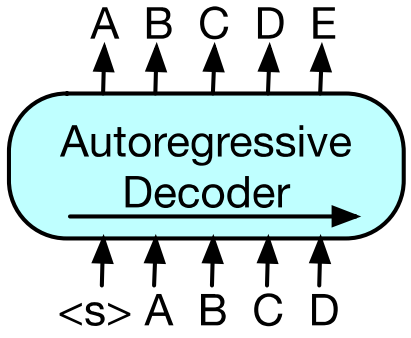


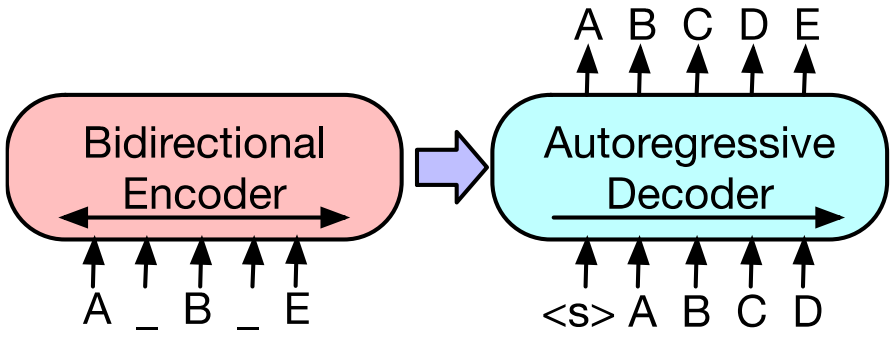
- Encode the prompt and generated text separately
- On the generation arm, perform cross attention with encoded prompt
- On generator, have self attention and cross attention layers, in addition to the FF element

BERT (Google)



GPT (Open AI)

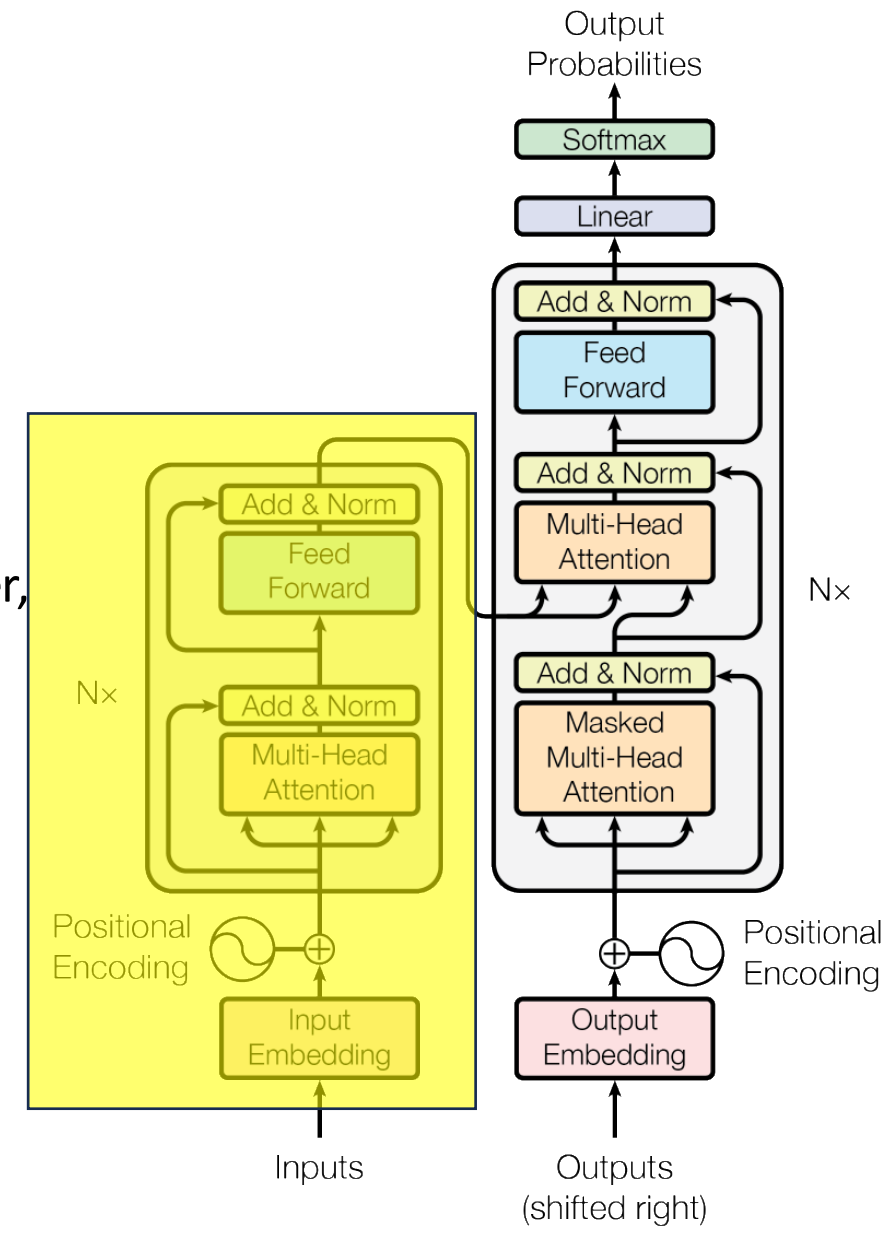




BART (Facebook)

BART

Like Original Transformer,
But Replace Encoder
with BERT



On the LSTM

- Long short-term memory was developed in the 1990s, and is an important class of recurrent neural networks
- Prior to the Transformer (2017), the LSTM was the dominant method for doing neural language modeling and related time-dependent analysis
- The Transformer yields comparable performance to the LSTM, but has important practical benefits, which is why today the LSTM is rarely used in practice
- Nevertheless, the LSTM is an important class of models, that a serious ML practitioner should know
- Knowledge of the LSTM will help explain the benefits of Transformers, and could help you on your future interviews
- Will cover later in the semester

Sequence Generation With Trained Model

- Given input x the probability of the generating sequence y, composed of N tokens is

$$p(y|x) = p(y_1|x)p(y_2|y_1, x)p(y_3|y_1, y_2, x) \dots p(y_N|y_1, y_2, \dots, y_{N-1}, x) = \prod_{i=1}^{N} P_\theta(y_i|x, y_{1:i-1})$$

- While we may understand this probabilistic equation, how do we actually generate the sequence y?

- Has N output tokens

- We have a softmax probability at each step

Sequence Generation With Trained Model

- Given input x and retrieved text z the probability of the generating sequence y, composed of N tokens is

$$p(y|x) = \prod_{i=1}^{N} P_\theta(y_i|x, y_{1:i-1})$$

- While we may understand this probabilistic equation, how do we actually generate the sequence $y = (y_1, \dots, y_N)$?

- Has N output tokens

- We have a softmax probability at each step; could just pick the most probable word at each step

- Generally does not generate particularly realistic text

Beam Search

- With Beam Search, we retain multiple (most probable) sequences
- The number of sequences we retain defines the size of the “beam”
- Consider an example with beam size two and consider the word dictionary:

A B C D E <end>

- When we yield most-probable sequence that has <end> as last token, then the decoder stops

Beam Search

- Consider an example with beam size two and consider the word dictionary:

A B C D E <end>

	Candidates	Probabilities	Selected
	A	0.4	A
	B	0.05	C
	C	0.5	
	D	0.025	
	E	0.025	
<start>	<end>	0	

Step 1; y_1

Beam Search

- Consider an example with beam size two and consider the word dictionary:

A B C D E <end>

	Candidates	Probabilities	Selected
	A	0.4	A
	B	0.05	C
	C	0.5	
	D	0.025	
	E	0.025	
<start>	<end>	0	

Step 1; y_1

Candidates	Probabilities	Selected
AA	0.02	AB
AB	0.3	CA
AC	0.0	
AD	0.1	
AE	0.02	
A<end>	0	
CA	0.4	
CB	0.02	
CC	0.02	
CD	0.1	
CE	0.02	
C<end>	0	

Step 2; y_1, y_2

Beam Search

- Consider an example with beam size two and consider the word dictionary:

A B C D E <end>

	Candidates	Probabilities	Selected
	A	0.4	A
	B	0.05	C
	C	0.5	
	D	0.025	
	E	0.025	
<start>	<end>	0	

Step 1; y_1

Candidates	Probabilities	Selected
AA	0.02	AB
AB	0.3	CA
AC	0.0	
AD	0.1	
AE	0.02	
A<end>	0	
CA	0.4	
CB	0.02	
CC	0.02	
CD	0.1	
CE	0.02	
C<end>	0	

Step 2; y_1, y_2

Beam Search

- Consider an example with beam size two and consider the word dictionary:

A B C D E <end>

Candidates	Probabilities	Selected
AA	0.02	AB
AB	0.3	CA
AC	0.0	
AD	0.1	
AE	0.02	
A<end>	0	
CA	0.4	
CB	0.02	
CC	0.02	
CD	0.1	
CE	0.02	
C<end>	0	

Step 2; y_1, y_2

Beam Search

- Consider an example with beam size two and consider the word dictionary:

A B C D E <end>

Candidates	Probabilities	Selected
AA	0.02	AB
AB	0.3	CA
AC	0.0	
AD	0.1	
AE	0.02	
A<end>	0	
CA	0.4	
CB	0.02	
CC	0.02	
CD	0.1	
CE	0.02	
C<end>	0	

Step 2; y_1, y_2

Candidates	Probabilities	Selected
ABA	0.0	ABD
ABB	0.025	CAD
ABC	0.0	
ABD	0.4	
ABE	0.025	
AB<end>	0.05	
CAA	0.1	
CAB	0.0	
CAC	0.025	
CAD	0.3	
CAE	0.025	
CA<end>	0.1	

Step 3; y_1, y_2, y_3

Beam Search

- Consider an example with beam size two and consider the word dictionary:

A B C D E <end>

Candidates	Probabilities	Selected
AA	0.02	AB
AB	0.3	CA
AC	0.0	
AD	0.1	
AE	0.02	
A<end>	0	
CA	0.4	
CB	0.02	
CC	0.02	
CD	0.1	
CE	0.02	
C<end>	0	

Step 2; y_1, y_2

Candidates	Probabilities	Selected
ABA	0.0	ABD
ABB	0.025	CAD
ABC	0.0	
ABD	0.4	
ABE	0.025	
AB<end>	0.05	
CAA	0.1	
CAB	0.0	
CAC	0.025	
CAD	0.3	
CAE	0.025	
CA<end>	0.1	

Step 3; y_1, y_2, y_3

. . .

Beam Search

- With Beam Search, we retain multiple (most probable) sequences
- The number of sequences we retain defines the size of the "beam"
- Consider an example with beam size two and consider the word dictionary:

A B C D E <end>

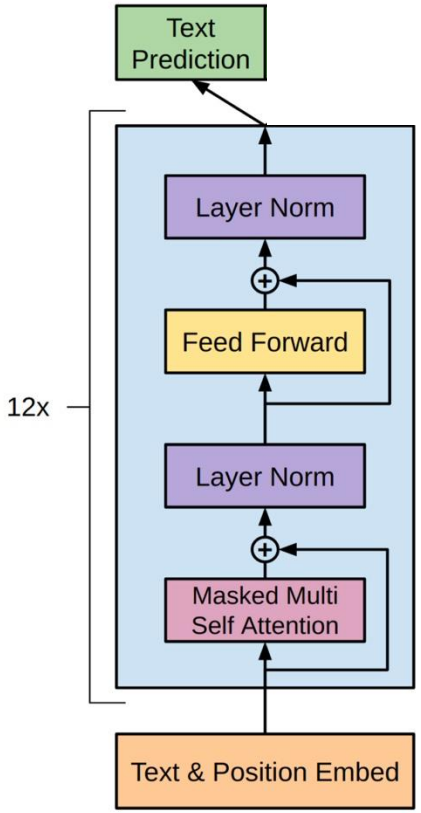
- When we yield most-probable sequence that has <end> as last token, then the decoder stops
- If beam size is B, then the computational cost is $B \times$ that of taking most-probable at each step (have to scan through softmax probabilities B times)

Retrieval Augmented Generation (RAG)

Sequence-to-Sequence (seq2seq) Language Models

- GPT-class LLMs may be viewed as seq2seq models

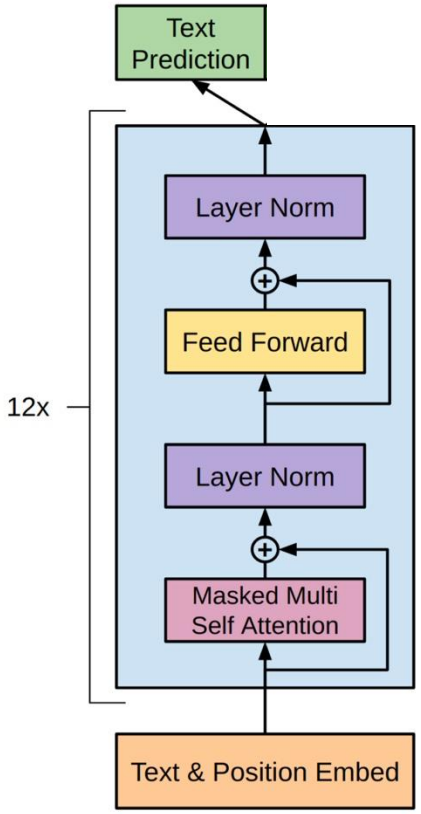
- An input sequence of tokens may be encoded to a vector associated with the last word in the sequence



Sequence-to-Sequence (seq2seq) Language Models

- GPT-class LLMs may be viewed as seq2seq models

- An input sequence of tokens may be encoded to a vector associated with the last word in the sequence



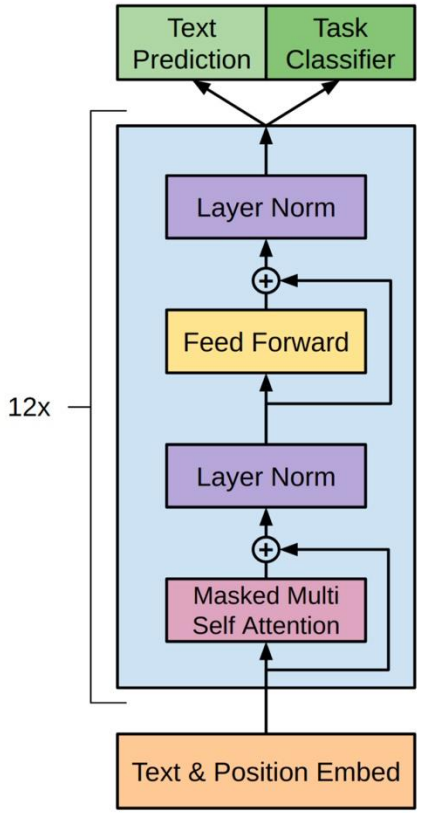
- Let the input sequence be represented by x and let $y_{1:i-1}$ represent the GPT-generated sequence thus far

- The Transformer-based (softmax) probability of the next word is represented as

$$p_\theta(y_i | x, y_{1:i-1})$$

- For input sequence x, one word at a time, we generate output sequence y (seq2seq)

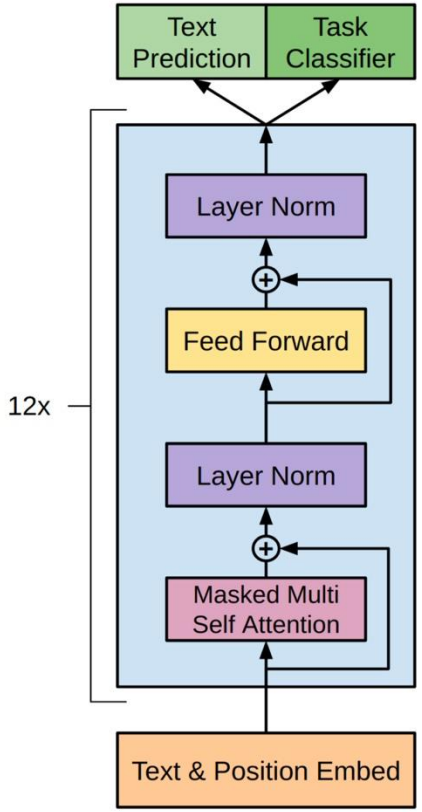
Sequence-to-Sequence (seq2seq) Language Models



- The parameters in the LLM capture memory of all knowledge that may be used for the generated sequence y
- Termed *parametric* memory
- We wish to augment this with nonparametric memory stored in a database of documents
- Wish to retrieve from the database the context most useful for generating output text y based on input x
- If z represents the retrieved context, appropriate for x, interested in the model

$$p_\theta(y_i|x, z, y_{1:i-1})$$

Sequence-to-Sequence (seq2seq) Language Models



Prompt/question

Retrieved text

$$p_\theta(y_i | x, z, y_{1:i-1})$$

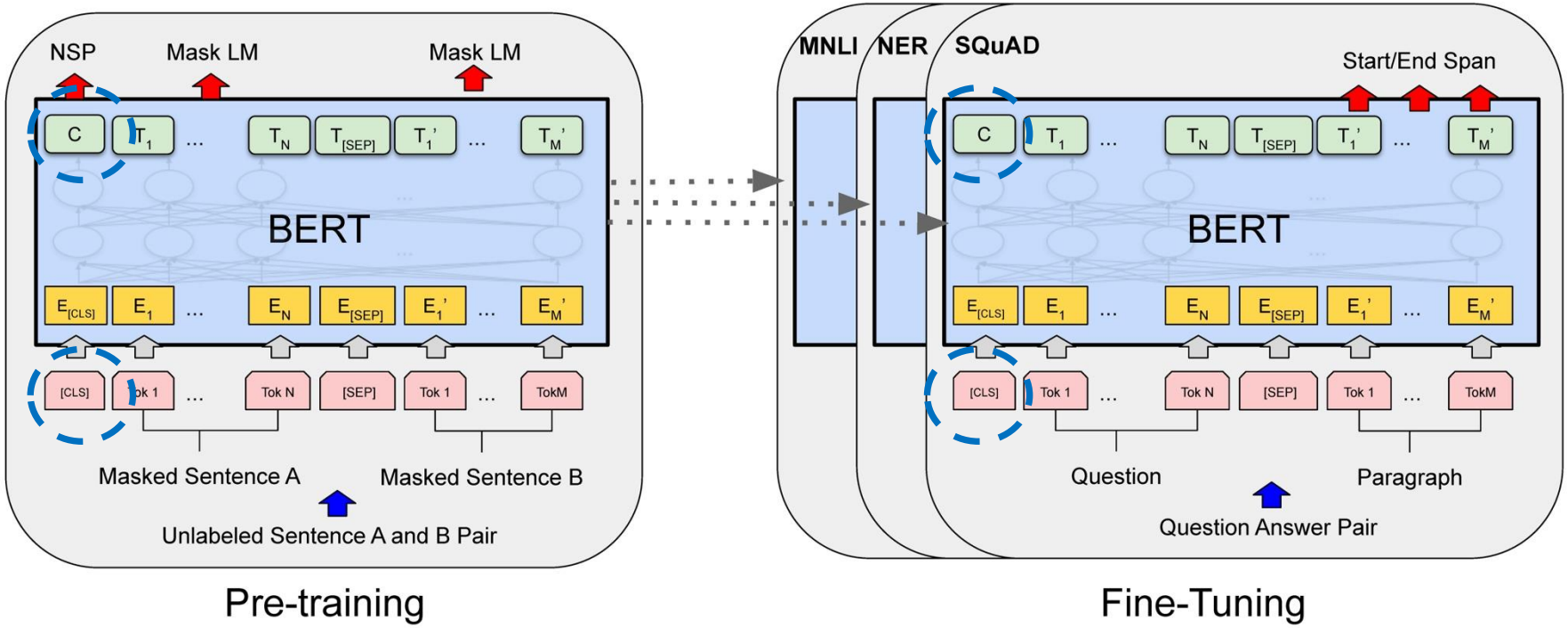
Next token

Text generated thus far

RAG

- Build a model capable of determining most relevant text from database given prompt x

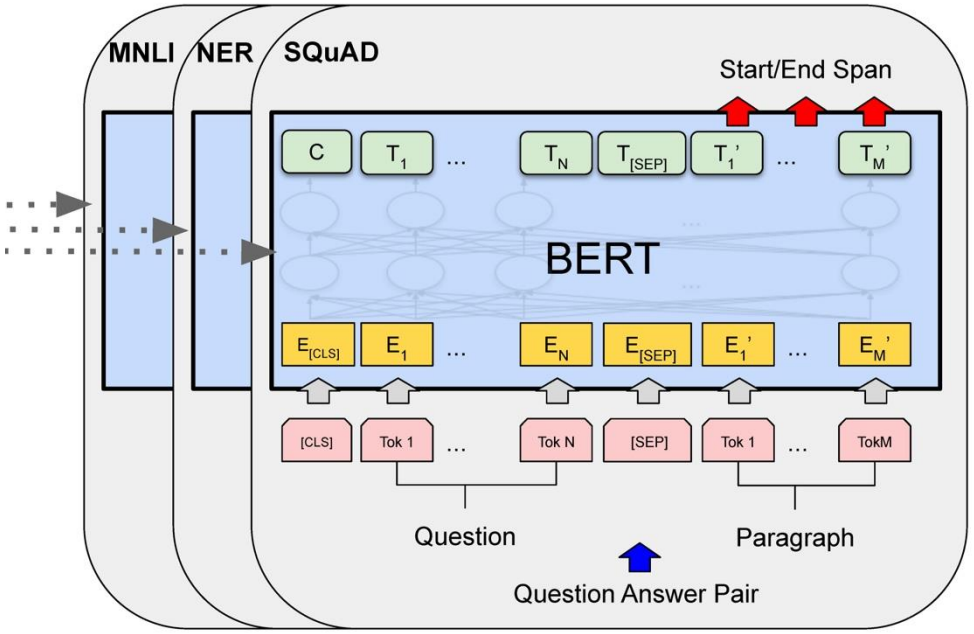
- One useful method: Use output from [CLS] input as a feature vector for each database entry and also for each prompt



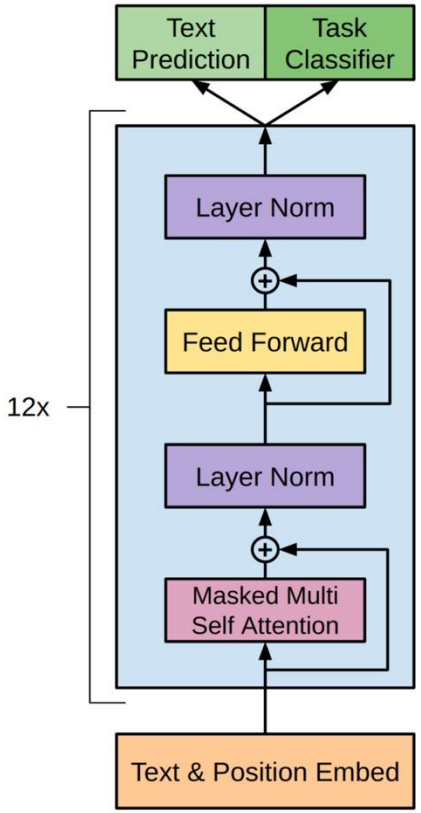
Two Models for RAG

- BERT for retrieval and GPT for seq2seq generation

- Fine-tune both together for RAG application



Fine-Tuning



Sequence Model

- Given input x and retrieved text z the probability of the generating sequence y, composed of N tokens is

$$p(y|x,z) = \prod_{i=1}^{N} P_\theta(y_i|x,z,y_{1:i-1})$$

- While we may understand this probabilistic equation, how do we actually generate the sequence y?

- Has N output tokens

- We have a softmax probability at each step

Sequence Model

- Given input x and retrieved text z the probability of the generating sequence y, composed of N tokens is

$$p(y|x,z) = \prod_{i=1}^{N} P_\theta(y_i|x,z,y_{1:i-1})$$

- While we may understand this probabilistic equation, how do we actually generate the sequence y?

- Has N output tokens

- We have a softmax probability at each step; could just pick the most probable word at each step

- Generally does not generate particularly realistic text

Beam Search

- With Beam Search, we retain multiple (most probable) sequences
- The number of sequences we retain defines the size of the "beam"
- Consider an example with beam size two and consider the word dictionary:

A B C D E <end>

- When we yield most-probable sequence that has <end> as last token, then the decoder stops
- If beam size is B, then the computational cost is $B \times$ that of taking most-probable at each step (have to scan through softmax probabilities B times)

BERT-Based Selection from Database of Documents

- Consider prompt x and element from database z

- Embed prompt and database element via BERT (using output above [CLS]):

$$d(z) = \mathrm{BERT}_d(z) \in R^M \qquad\qquad q(x) = \mathrm{BERT}_q(x) \in R^M$$

- The probability of z matching x satisfies

$$p_\eta(z|x) \propto \exp[d(z) \cdot q(x)/\eta]$$

- In general, we train two different BERT models, one for question and the other for the database

BERT-Based Selection from Database of Documents

$$p_\eta(z|x) \propto \exp[d(z) \cdot q(x)/\eta]$$

- Would be standard softmax if we computed this probability using all elements in the database
- We only consider the K most probable elements in the database, and normalize accordingly
- Let $S_q(x)$ represent the set of K most probable elements from the database for prompt x

$$p_\eta\big(z \in S_q(x)\big|x\big) = \frac{\exp[d\big(z \in S_q(x)\big) \cdot q(x)/\eta]}{\sum_{z' \in S_q(x)} \exp[d(z')) \cdot q(x)/\eta]}$$

- Sum involves K elements $z' \in S_q(x)$

In Practice, Two Approaches for RAG Implementation

- **RAG sequence**:
  - For text z retrieved from the database, generate text

$$p(y|z,x) = \prod_{i=1}^{n} p(y_i|z,x,y_{1:i-1})$$

  - For each pair (x,z) generate candidate text y, and select the most probable set of possible generated text via beam search
  - Do this for all candidate text from database, $z \in S_{q(x)}$
  - For all candidate generated text, compute

$$p(y|x) = \sum_{z \in S_q(x)} p_\eta(z|x)p(y|x,z)$$

In Practice, Two Approaches for RAG Implementation

- **RAG token**:
  - With RAG-Sequence, we average over $z \in S_{q(x)}$ *after* generating all possible complete generated text
  - With RAG-Token, we average over $z \in S_{q(x)}$ when selecting each token

$$p(y_i|x, y_{1:i-1}) = \sum_{z \in S_{q(x)}} p_\gamma(z|x) p(x, z, y_{1:i-1})$$

  - All candidate generated at once by beam search, token by token, averaging over $p_\gamma(z|x)$ at each step
  - Same set of prior generations $y_{1:i-1}$ used for all $z \in S_{q(x)}$

RAG-Token Model

- Recall the generative process, now conditioned on prompt x along with text from a database, z

$$p(y|x,z) = \prod_{i=1}^{N} P_\theta(y_i|x,z,y_{1:i-1})$$

- If we have

$$p_\eta\big(z \in S_q(x)\big|x\big) = \frac{\exp[d\big(z \in S_q(x)\big) \cdot q(x)/\eta]}{\sum_{z' \in S_q(x)} \exp[d(z')) \cdot q(x)/\eta]}$$

- At every step of beam decoding we work with $p(y_i|x,z,y_{1:i-1})$ and therefore considering the K selected elements from the database, we have

$$p(y_i|x,y_{1:i-1}) = \sum_{z \in S_q(x)} p_\eta(z|x) p(y_i|x,z,y_{1:i-1})$$

RAG-Token Model

- Our conditional model for generated text given the prompt x is

$$p(y_i|x, y_{1:i-1}) = \sum_{z \in S_q(x)} p_\eta(z|x) p(y_i|x, z, y_{1:i-1})$$

- From this the full probability of the total generated text is

$$p(y|x) = p(y_1, \dots, y_n|x) = \prod_{i=1}^{n} p(y_i|x, y_1, \dots, y_{i-1})$$

- We utilize the above $p(y|x)$ when evaluating generated text for beam search

- Via the sum $\sum_{z \in S_q(x)} p_\eta(z|x) p(y_i|x, z, y_{1:i-1})$ we "marginalize out" the set of texts z taken from the database

$$p(y_i|x, y_{1:i-1}) = \sum_{z \in S_q(x)} p_\eta(z|x) p(y_i|x, z, y_{1:i-1})$$

Text from database

Prob of match between prompt x database element x

$$p(y_i|x, y_{1:i-1}) = \sum_{z \in S_q(x)} \overbrace{p_\eta(z|x)}^{\text{Prob of match between prompt } x \text{ database element } x} p(y_i|x, z, y_{1:i-1})$$

Text from database

Prob of match between prompt x database element x

$$p(y_i|x, y_{1:i-1}) = \sum_{z \in S_q(x)} p_\eta(z|x) p(y_i|x, z, y_{1:i-1})$$

K most probable database members

Text from database

Prob of match between prompt x database element x

$$p(y_i|x, y_{1:i-1}) = \sum_{z \in S_q(x)} p_\eta(z|x) p(y_i|x, z, y_{1:i-1})$$

K most probable database members

Text from database

- Can be implemented by beam search

- The probability $p_\eta(z|x)$ is considered in the generation of each token y_i to marginalize out database selections

RAG Token

Prob of match between prompt x database element x

$$p(y_i|x, y_{1:i-1}) = \sum_{z \in S_q(x)} p_\eta(z|x) p(y_i|x, z, y_{1:i-1})$$

K most probable database members

Text from database

- Can be implemented by beam search

- The probability $p_\eta(z|x)$ is considered in the generation of each token y_i to marginalize out database selections

$$p_{RAG-Token}(y|x) = \prod_{i=1}^{n} \sum_{z \in Z_q(x)} p_\eta(z|x) p(y_i|x, z, y_{1:i-1})$$

RAG-Token Model

$$p_{RAG-Token}(y|x) = \prod_{i=1}^{n} \sum_{z \in \mathrm{Z}_q(x)} p_\eta(z|x) p(y_i|x, z, y_{1:i-1})$$

- In the RAG-Token model, we average over the probability of K database entries for each token

- Beam search applied using probabilities manifested after implementing $\sum_{z \in \mathrm{Z}_q(x)} p_\eta(z|x) p(y_i|x, z, y_{1:i-1}$

RAG-Sequence Model

- In the RAG-Token model, we average over the probability of K database entries for each token

- For sequence, we generate a sequence for each of the K database entries at the end:

$$p_{RAG-Sequence}(y|x) = \sum_{z \in S_q(x)} p_\eta(z|x) \prod_{i=1}^{n} p_\theta(y_i|x, z, y_{1:i-1})$$

- Via the beam search, we have a set of possible sequences Y, that survive the beam search over all $z \in S_q(x)$

- Challenge: Not all sequences in Y may have been associated with all $z \in S_q(x)$

RAG-Sequence Model

- In the RAG-Token model, we average over the probability of K database entries for each token

- For sequence, we generate a sequence for each of the K database entries at the end:

$$p_{RAG-Sequence}(y|x) = \sum_{z \in S_q(x)} p_\eta(z|x) \prod_{i=1}^{n} p_\theta(y_i|x, z, y_{1:i-1})$$

- Via the beam search, we have a set of possible sequences Y, that survive the beam search over all $z \in S_q(x)$

- Challenge: Not all sequences in Y may have been associated with all $z \in S_q(x)$

- For all $y \in Y$ not addressed by some $z \in S_q(x)$, run Transformer to compute $\prod_{i=1}^{n} p_\theta(y_i|x, z, y_{1:i-1})$

RAG-Sequence Model

$$p_{RAG-Sequence}(y|x) = \sum_{z \in S_q(x)} p_\eta(z|x) \prod_{i=1}^{n} p_\theta(y_i|x, z, y_{1:i-1})$$

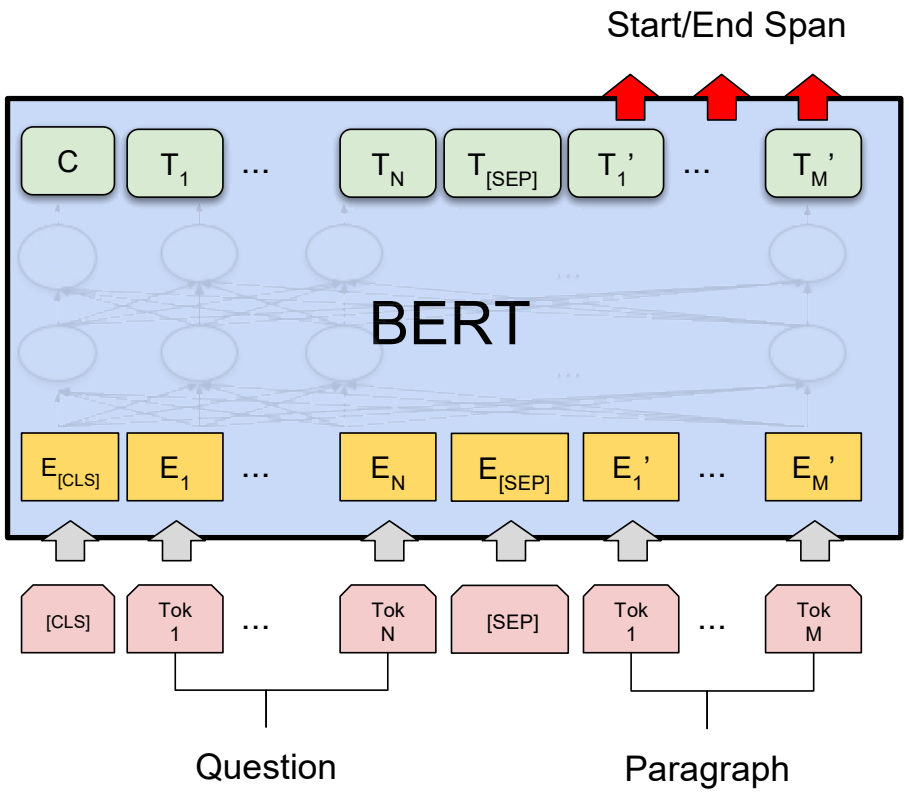
- For all $y \in Y$ not addressed by some $z \in S_q(x)$, run Transformer to compute $\prod_{i=1}^{n} p_\theta(y_i|x, z, y_{1:i-1})$

- Could be computationally expensive, depending on size of Y so approximations may be used

RAG Training

- Consider a pretrained BERT model for retrieval, and a pretrained language model (e.g., GPT) for generation
- These models are now fine-tuned for this particular task
- Assume given a set of training data $D_T = \{(x_i, y_i)\}_{i=1,N}$ of prompts and desired generated text
- Train the BERT model and GPT simultaneously, to maximize the likelihood of D_T
- The database could be very large, and therefore it is expensive to fine-tune $\mathrm{BERT}_d(z)$
- Therefore, in fine-tuning, update parameters of $\mathrm{BERT}_q(x)$ and $p_\theta(y|x, z)$

Another RAG Applications: Find Answers Within Text



Q&A RAG Setup

- User asks a question for which there is a factual answer that may be pulled from text
- Given the question, find candidate text passages in which the answer is likely found
- Given those text passages, use the BERT down-stream methodology to find the span of the answer within the pulled text
- Assume that the BERT find-the-answer model has been trained
- Consider how to find appropriate text candidates from a database
- Capacity to pull appropriate text from a database can then be used for other applications, like the text generation discussed above

Q&A RAG Setup

- For training the text-search model, assume we have a large database of questions and associated text in which the answer may be found

- Let $\{q_n\}_{n=1,N}$ represent the set of questions for which there are factual answers

- Let $\{t_n\}_{n=1,N}$ be the associated database of text in which the answers may be found

- Encode with BERT, based on the output above the [CLS] token (two different BERT models)

$$T_n = \mathrm{BERT}_t(t_n) \in R^M \qquad Q_n = \mathrm{BERT}_q(q_n) \in R^M$$

$$p_\eta(t_m|q_n) \propto \exp[T_m \cdot Q_n/\eta]$$

Q&A RAG Setup

- Train both BERT models such that appropriate question-text pair is highly probable and the wrong match has low probability

$$T_n = \mathrm{BERT}_t(t_n) \in R^M \qquad Q_n = \mathrm{BERT}_q(q_n) \in R^M$$

$$p_\eta(t_m|q_n) \propto \exp[T_m \cdot Q_n/\eta]$$

- Supplement the set of answers with a large set of text blocks that are aligned topically with the questions, but do not contain the answers

- Once such a database-search system is setup, the separately trained BERT system can extract the location of answers from appropriate text

Open Questions

- GPT and BERT style Transformers are powerful models
- For text generation, GPT class models are necessary
- Challenge: With cross-entropy loss, we train them to predict next word
- If we generate text by taking the most probable output at each step, text is typically not natural
- Beam search is a significant improvement for generation of realistic text
- Still: The model is not trained such that it is aligned with human preferences; no reason to believe that cross entropy loss with one word at a time will yield text aligned with human preferences, even with beam search

Leveraging Human Feedback

- Modern Transformers give opportunity to provide feedback
- Thumb up or down
- People also often write ”thank you” when conversing with an LLM
- One can also examine the human interaction to detect dissatisfaction with the generated text
- Here we assess the entire generated text, not just one word at a time
- How do we develop a loss function that accounts for human feedback on entire text?

Alignment with Reinforcement Learning

- Reinforcement learning is like learning how to ride a bike just based on feedback on how you have done so far
- If you do some actions that keep the bike moving, and you atop it: Do more of that.
- If you do something that leads to an abrupt fall: Don’t do that again.
- Through continual trial and error, based on feedback, you learn to ride a bike, with no formal instructions
- This is reinforcement learning; reinforce actions that yield high rewards, based on feedback
- Use human feedback on LLM outputs, and based on that refine and continually improve the model

Reinforcement Learning

- Reinforcement learning is a field of long-term study in machine learning
- Build on that, and tailor it to the special characteristics of language
- In addition to being of value for improving LLMs, the study of RL is of general interest in machine learning
- We will next study RL in the context of aligning LLM outputs with the desires of humans
- Further, we will examine RL to teach an LLM to perform chain-of-thought reasoning